

Intelligent Cognitive Alarm Platform

Milestone 1 Report

Prepared by: Komala from team 5

Table of Contents

1. Project Description	3
1.1 Module Scope	3
1.2 Captured User Information	3
2. Environment Setup	3
2.1 Tech Stack	3
3. Data Model	4
3.1 Core Entities	4
4. Proposed System	4
4.1 Data Flow	4
5. Implementation Pipeline	4
5.1 Backend	4
5.2 Frontend	4
6. Verification & Testing	5
7. API Implementation — Swagger Documentation	5
7.1 GET /profile/ — Retrieve All Profiles	5
7.2 POST /profile/ — Create Profile	6
7.3 PUT /profile/{profile_id} — Update Profile	6
7.4 DELETE/profile -Delete profile	6
8. Conclusion	7

1. Project Description

The Intelligent Cognitive Alarm Platform is an AI-powered system that helps users build consistent wake-up habits by requiring them to solve personalized cognitive challenges — puzzles, riddles, memory tasks, logic problems, and math exercises — before an alarm can be dismissed. The platform adapts challenge difficulty based on each user's wake-up performance, snooze behavior, sleep schedule, and cognitive engagement, with the broader goal of improving productivity, reducing oversleeping, and encouraging healthier sleep routines.

This report covers Module 2: User Profile & Habit Management, the component responsible for capturing user information, sleep schedules, wake-up goals, and habit preferences that downstream modules — the Cognitive Challenge Engine, Adaptive Difficulty Engine, and Behavioral Analytics Engine — depend on.

1.1 Module Scope

- User profile creation and management
- Sleep schedule configuration (bedtime, wake-up time, time zone)
- Wake-up goal tracking
- Productivity goal setup
- Habit profile and preference management

1.2 Captured User Information

- Preferred Wake-up Time
- Sleep Duration
- Time Zone
- Productivity Goals
- Difficulty Preference
- Habit Preferences

2. Environment Setup

Module 2 was developed as part of the platform's microservices layer, sitting behind the API Gateway and interacting with the primary relational data store.

2.1 Tech Stack

Layer	Technology
Backend Framework	Python, FastAPI
Primary Database	PostgreSQL
Frontend (Web)	React.js
Frontend (Mobile)	React Native (Android/iOS)
Authentication	JWT, OAuth2, Role-Based Access Control
Containerization	Docker, Docker Compose
Version Control / CI	Git, GitHub, GitHub Actions
API Testing	Swagger

The service is stateless at the API layer, with user profile, sleep-schedule, and habit-preference records persisted in PostgreSQL and exposed to the rest of the platform through versioned REST endpoints behind the FastAPI gateway.

3. Data Model

The module's schema is designed to separate identity data (owned by the Authentication module) from behavioral and preference data (owned by Module 2), so that habit-related fields can evolve independently.

3.1 Core Entities

- User Profile — name, preferred wake-up time, time zone, difficulty preference
- Sleep Schedule — bedtime, target wake time, sleep duration goal, recurrence pattern
- Productivity Goals — goal type, target metric, target date
- Habit Preferences — preferred challenge types, notification preferences, habit tracking opt-ins

These entities feed the Habit Scoring Engine (Module 8) and Adaptive Difficulty Engine (Module 5) downstream, and are read by the Alarm Scheduling System when generating a user's next scheduled alarm.

4. Proposed System

Module 2 operates as a supporting service within the platform's microservices layer, feeding structured user context to the modules responsible for challenge generation, difficulty adaptation, and behavioral analytics.

4.1 Data Flow

1. Registration & Onboarding: A new user, authenticated via the User Authentication module, is prompted to complete their profile.
2. Profile Creation: The user submits wake-up time, sleep duration, time zone, and productivity goals through the React Native app or web dashboard.
3. Habit Preference Capture: The user selects preferred challenge types and difficulty preference, stored as structured habit-preference records.
4. Persistence: FastAPI validates and writes the profile, sleep schedule, and preferences to PostgreSQL.
5. Downstream Consumption: The Alarm Scheduling System, Cognitive Challenge Engine, and Adaptive Difficulty Engine read this profile data to personalize alarms and challenges.

5. Implementation Pipeline

Milestone 1 focused on establishing the module end-to-end: backend endpoints, database schema, validation, and the corresponding frontend screens.

5.1 Backend

- FastAPI routes for creating, retrieving, and updating user profiles
- PostgreSQL schema and migrations for profile, sleep-schedule, and habit-preference tables
- Request/response validation using Pydantic models
- Role-based access control integration so only the authenticated user (or an authorized Wellness Coach/Administrator) can view or modify a given profile

5.2 Frontend

- React Native onboarding flow for initial profile setup
- Sleep schedule and wake-up goal configuration screens
- Productivity goal setup screen

- Habit preference selection screen (challenge types, difficulty preference)
- Profile edit/view screen reflecting saved data from the backend

6. Verification & Testing

Module 2 was verified against the Milestone 1 evaluation criteria for user profile management: profile creation, retrieval, and update operations were exercised through Postman against the FastAPI endpoints, and the corresponding React Native screens were validated against the live backend to confirm that saved profile and habit data is correctly reflected in the UI.

- Endpoint-level testing (create/read/update profile, sleep schedule, and preferences)
- Schema validation for required fields (wake-up time, time zone, sleep duration)
- Access control checks confirming a user cannot read or modify another user's profile
- Frontend-to-backend integration check across onboarding and edit flows

7. API Implementation — Swagger Documentation

The User Profile endpoints were implemented in FastAPI and verified directly against the auto-generated Swagger UI (/docs). The three core operations — retrieve, create, and update a profile — are shown below with live request/response payloads.

7.1 GET /profile/ — Retrieve All Profiles

Returns the stored profile records, including preferred wake-up time, sleep duration, time zone, productivity goal, difficulty preference, and habit preference for each user.

Intelligent Cognitive Alarm Platform 1.0.0 OAS 3.1

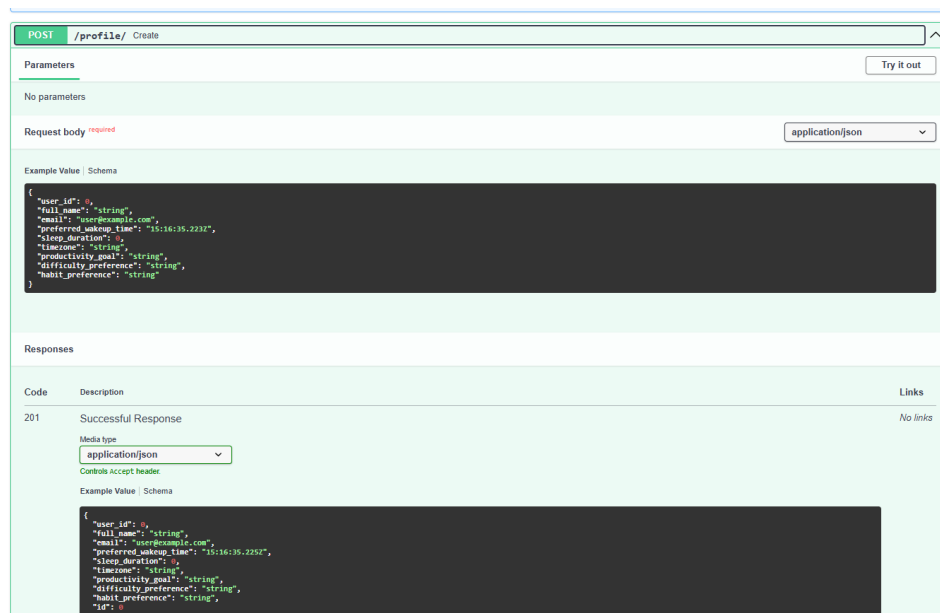
/openapi.json

The screenshot displays the Swagger UI for the 'Intelligent Cognitive Alarm Platform' (version 1.0.0, OAS 3.1). The endpoint being viewed is 'GET /profile/'. The 'Parameters' section indicates 'No parameters'. Below this, there is an 'Execute' button and a 'Clear' button. The 'Responses' section shows a '200' status code and a 'Response body' containing a JSON object with the following structure:

```
{
  "user_id": 1,
  "full_name": "Komal Karam",
  "email": "komal@icaps.com",
  "preferred_wakeup_time": "06:30:00",
  "sleep_duration": 8,
  "timezone": "Asia/Kolkata",
  "productivity_goal": "study 6 hours",
  "difficulty_preference": "medium",
  "habit_preference": "early wakeup",
  "id": 1
}
```

7.2 POST /profile/ — Create Profile

Accepts a new profile payload (full name, email, preferred wake-up time, sleep duration, time zone, productivity goal, difficulty preference, habit preference) and persists it, returning a 201 response with the created record.



Swagger UI for the **POST /profile/ Create** endpoint. The interface shows the method and path, a 'Try it out' button, and a dropdown for the request body media type set to 'application/json'. The 'Parameters' section indicates 'No parameters'. The 'Request body' section is marked as 'required' and shows an example JSON payload. The 'Responses' section shows a '201 Successful Response' with a dropdown for the media type set to 'application/json' and an example JSON response.

Parameters

No parameters

Request body required

application/json

Example Value | **Schema**

```
{
  "user_id": 0,
  "full_name": "string",
  "email": "user@example.com",
  "preferred_wakeup_time": "15:16:35.223Z",
  "sleep_duration": 0,
  "timezone": "string",
  "productivity_goal": "string",
  "difficulty_preference": "string",
  "habit_preference": "string"
}
```

Responses

Code	Description	Links
201	Successful Response	No links

Media type: application/json

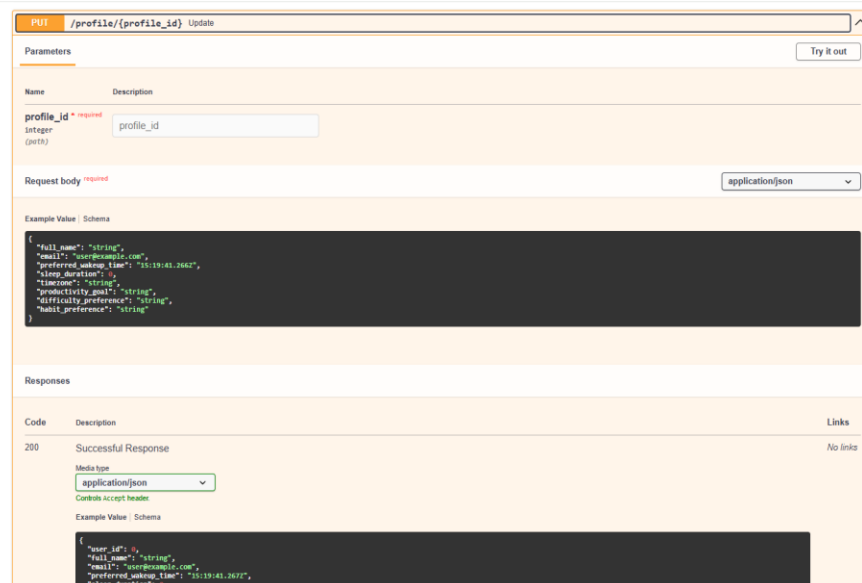
Controls: Accept header

Example Value | **Schema**

```
{
  "user_id": 0,
  "full_name": "string",
  "email": "user@example.com",
  "preferred_wakeup_time": "15:16:35.223Z",
  "sleep_duration": 0,
  "timezone": "string",
  "productivity_goal": "string",
  "difficulty_preference": "string",
  "habit_preference": "string",
  "id": 0
}
```

7.3 PUT /profile/{profile_id} — Update Profile

Updates an existing profile by profile_id, accepting the same field set as creation and returning a 200 response with the updated record.



Swagger UI for the **PUT /profile/{profile_id} Update** endpoint. The interface shows the method and path, a 'Try it out' button, and a dropdown for the request body media type set to 'application/json'. The 'Parameters' section shows a required path parameter 'profile_id' of type 'integer'. The 'Request body' section is marked as 'required' and shows an example JSON payload. The 'Responses' section shows a '200 Successful Response' with a dropdown for the media type set to 'application/json' and an example JSON response.

Parameters

Name	Description
profile_id * required	profile_id

integer (path)

Request body required

application/json

Example Value | **Schema**

```
{
  "full_name": "string",
  "email": "user@example.com",
  "preferred_wakeup_time": "15:19:41.766Z",
  "sleep_duration": 0,
  "timezone": "string",
  "productivity_goal": "string",
  "difficulty_preference": "string",
  "habit_preference": "string"
}
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls: Accept header

Example Value | **Schema**

```
{
  "user_id": 0,
  "full_name": "string",
  "email": "user@example.com",
  "preferred_wakeup_time": "15:19:41.267Z",
  "sleep_duration": 0
}
```

7.4 DELETE /profile/{profile_id} — Delete Profile

DELETE /profile/{profile_id} Delete

Parameters

Name	Description
profile_id * required	
integer	10
(path)	

Execute

Clear

Responses

Curl

curl -X 'DELETE' \
'http://127.0.0.1:8000/profile/10' \
-H 'accept: application/json'

Request URL

http://127.0.0.1:8000/profile/10

Server response

Code	Details
200	Response body { "message": "Profile deleted successfully" } Response headers content-length: 42 content-type: application/json date: Fri, 10 Jul 2026 16:49:49 GMT server: uvicorn

Responses

Code	Description	Links
200	Successful Response	No links

8. Conclusion

Module 2 establishes the User Profile & Habit Management foundation of the Intelligent Cognitive Alarm Platform: capturing wake-up preferences, sleep schedules, productivity goals, and habit preferences through a FastAPI/PostgreSQL backend and React Native frontend. With this data model and set of endpoints in place, the platform is positioned to move into Milestone 2, where the Cognitive Challenge Engine and Wake-Up Verification module will begin consuming these habit preferences to personalize challenges.